ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA ĐIỆN - ĐIỆN TỬ

BÁO CÁO THÍ NGHIỆM
MÔN: THIẾT KẾ VI MẠCH

LAB 1: Hardware Description and Digital Design Verification

Lớp: L03 - NHÓM: 5
GVHD: ThS. Nguyễn Trung Hiếu

MSSV	Họ và tên	Tỉ lệ phân công
2311267	Vũ Nhật Huy	100%
2313355	Nguyễn Minh Thuận	100%
2313222	Thái Đức Thiên	100%

Thành phố Hồ Chí Minh, tháng 3 năm 2026

Mục lục

1. Experiment 1: Combinational Circuits - Adder/Subtractor **2**
1.1 Sơ đồ thiết kế của mạch. 2
1.2 Mô tả mạch thiết kế bằng ngôn ngữ System Verilog HDL. 2
1.3 Hoàn thành giản đồ xung dựa theo thiết kế trên. 3
1.4 Viết Testbench kiểm chứng thiết kế dựa trên giản đồ xung. 4

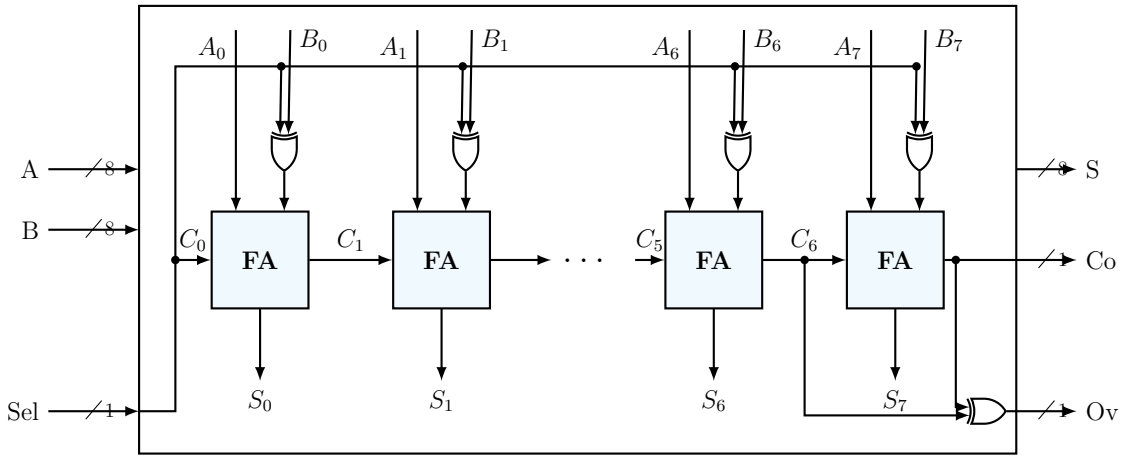
2. Experiment 2: Sequential Circuits - Accumulator **5**
2.1 Sơ đồ thiết kế của mạch. 5
2.2 Mô tả mạch thiết kế bằng ngôn ngữ System Verilog HDL. 7
2.3 Hoàn thành giản đồ xung dựa theo thiết kế trên. 8
2.4 Viết Testbench kiểm chứng thiết kế dựa trên giản đồ xung. 9
2.5 So sánh giữa hai thiết kế mạch 10

3. Experiment 3: FSM Implementation, Logic-Expression Style **12**
3.1 Phương trình trạng thái và ngõ ra 12
3.2 Mô tả mạch thiết kế bằng ngôn ngữ System Verilog HDL. 12
3.3 Hoàn thành giản đồ xung dựa theo thiết kế trên. 14
3.4 Viết Testbench kiểm chứng thiết kế dựa trên giản đồ xung. 15

4. Experiment 4: FSM Implementation, Behavioral Style **16**
4.1 Mô tả mạch thiết kế bằng ngôn ngữ System Verilog HDL. 16
4.2 Hoàn thành giản đồ xung dựa theo thiết kế trên. 17
4.3 Viết Testbench kiểm chứng thiết kế dựa trên giản đồ xung. 18

1. Experiment 1: Combinational Circuits - Adder/Subtractor

1.1 Sơ đồ thiết kế của mạch.



Hình 1: *Sơ đồ khối chi tiết mạch Adder/Subtractor 8-bit cấu trúc Ripple Carry với logic Overflow.*

Cách hoạt động của mạch:

- Mạch thực hiện phép toán cộng hoặc trừ 2 toán tử 8 bit A và B dựa vào tín hiệu lựa chọn Sel, khi Sel=0 thì mạch thực hiện phép cộng và khi Sel=1 thì mạch thực hiện phép trừ.
- Phát hiện Overflow: overflow được xác định bằng phép toán XOR giữa bit nhớ vào và bit nhớ ra của bit dấu: carry[7] XOR carry[6] như trên hình 1.
- Phát hiện Carry: Carry cuối cùng (tín hiệu carry[7] được lưu trữ và xuất ra ngoài qua Co).

1.2 Mô tả mạch thiết kế bằng ngôn ngữ System Verilog HDL.

Listing 1: `00_src/fa.sv`

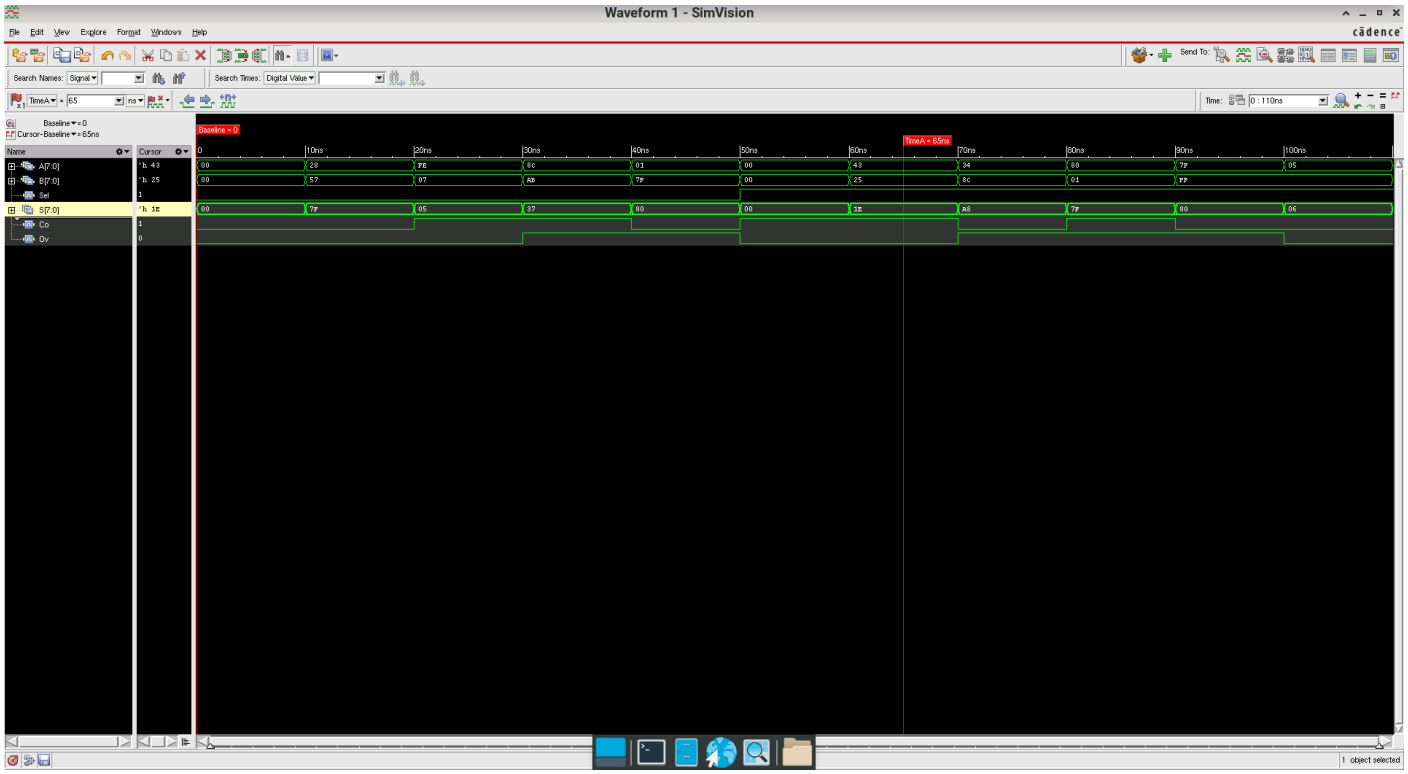
```
`timescale 1ns/1ns

module fa(input logic a,
      input logic b,
      input logic ci,
      output logic s,
      output logic co);
  assign s = a^b^ci;
  assign co=a&b | ci&a | ci&b;
endmodule
```

Listing 2: `00_src/tn1.sv`

```
`timescale 1ns/1ns

module tn1 (
    input  logic [7:0] A,
    input  logic [7:0]  B,
    input  logic       Sel,
    output logic [7:0] S,
    output logic       Co,
    output logic       Ov
);
    logic [7:0] temp;
    fa fa0 (.a(A[0]),.b(B[0] ^ Sel),.ci(Sel)     ,.s(S[0]),.co(temp[0]));
```

```
    fa fa1 (.a(A[1]),.b(B[1] ∧ Sel),.ci(temp[0]),.s(S[1]),.co(temp[1]));
    fa fa2 (.a(A[2]),.b(B[2] ∧ Sel),.ci(temp[1]),.s(S[2]),.co(temp[2]));
    fa fa3 (.a(A[3]),.b(B[3] ∧ Sel),.ci(temp[2]),.s(S[3]),.co(temp[3]));
    fa fa4 (.a(A[4]),.b(B[4] ∧ Sel),.ci(temp[3]),.s(S[4]),.co(temp[4]));
    fa fa5 (.a(A[5]),.b(B[5] ∧ Sel),.ci(temp[4]),.s(S[5]),.co(temp[5]));
    fa fa6 (.a(A[6]),.b(B[6] ∧ Sel),.ci(temp[5]),.s(S[6]),.co(temp[6]));
    fa fa7 (.a(A[7]),.b(B[7] ∧ Sel),.ci(temp[6]),.s(S[7]),.co(temp[7]));
    assign Co = temp[7];          // Carry out
    assign Ov = temp[7] ∧ temp[6];      // Overflow
endmodule
```

1.3 Hoàn thành giản đồ xung dựa theo thiết kế trên.



Hình 2: *Giản đồ thời gian mô phỏng mạch Adder/Subtractor.*

Giải thích:

- **Trường hợp Sel = 0 (Phép cộng):**
  - Không tràn: A = 0x23, B = 0x57 → S = 0x7A. Kết quả nằm trong khoảng biểu diễn, Co = 0, Ov = 0.
  - Tràn số không dấu (Carry): A = 0xFE (254), B = 0x07 (7) → S = 0x05. Tổng vượt quá 255 (tràn 8-bit), hệ thống sinh cờ nhớ nên Co = 1. Hai số dương cộng lại không gây sai dấu nên Ov = 0.
  - Tràn số có dấu (Overflow):
    * Âm + Âm = Dương: A = 0x8C (-116), B = 0xAB (-85) → Kết quả S = 0x37 (+55). Do cộng 2 số âm lại ra số dương (sai logic dấu) nên cờ tràn Ov = 1. Giá trị tràn 8-bit nên Co = 1.

  * Dương + Dương = Âm: A = 0x01 (+1), B = 0x7F (+127) → Kết quả S = 0x80 (-128). Do cộng 2 số dương lại ra số âm nên cờ tràn Ov = 1. Co = 0.

- **Trường hợp Sel = 1 (Phép trừ):**
  - Phép trừ thông thường (A ≥ B): A = 0x43, B = 0x25 → S = 0x1E. Trong mạch bù 2, Co = 1 biểu thị phép trừ không cần mượn.
  - Phép trừ có mượn (A < B): A = 0x34, B = 0xBC → S = 0x78. Kết quả sinh ra Co = 0, biểu thị phép trừ cần mượn.
  - Tràn số có dấu (Overflow):
    * Âm - Dương = Dương: A = 0x80 (-128), B = 0x01 (+1) → S = 0x7F (+127). Bị sai dấu nên Ov = 1.
    * Dương - Âm = Âm: A = 0x7F (+127), B = 0xFF (-1) → S = 0x80 (-128). Bị sai dấu nên Ov = 1.

1.4 Viết Testbench kiểm chứng thiết kế dựa trên giản đồ xung.

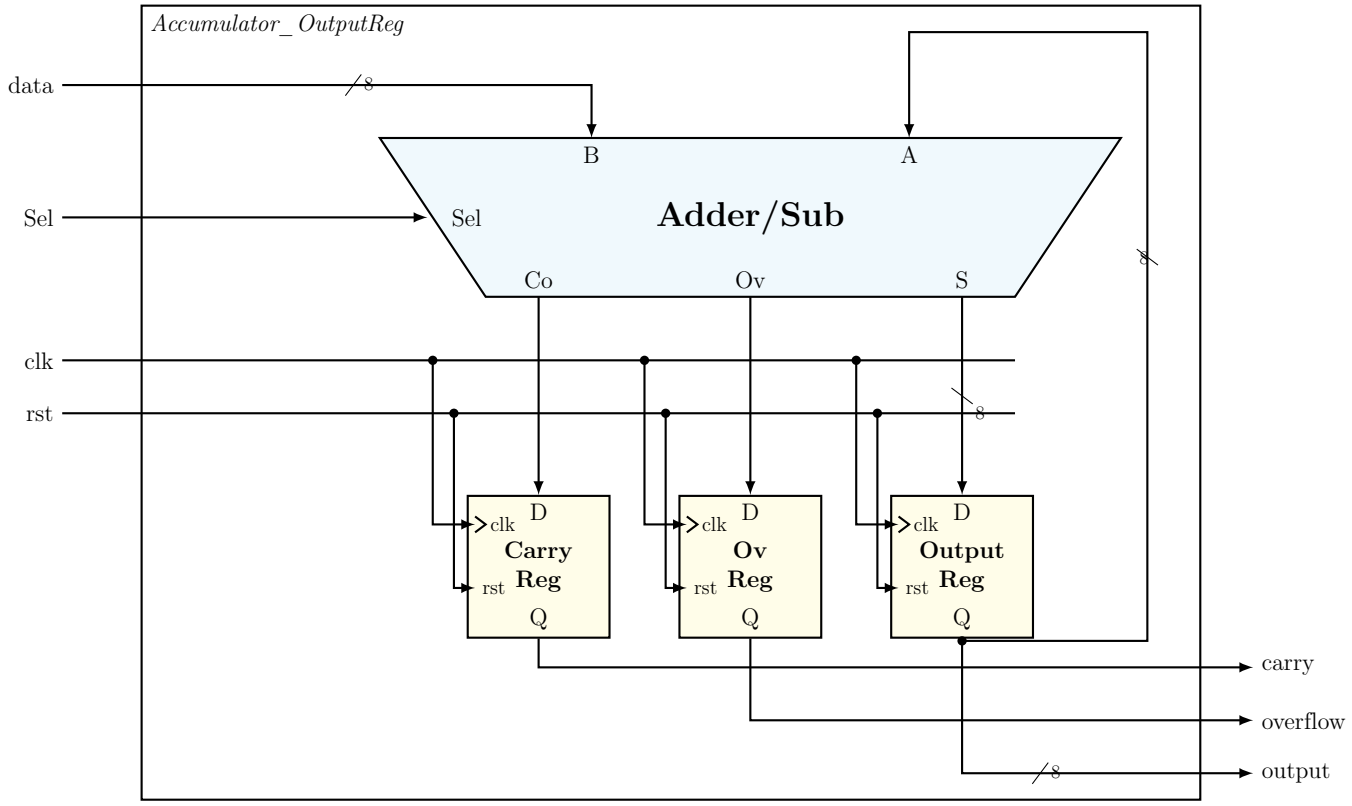
Listing 3: `01_tb/tb_tn1.sv`

```
`timescale 1ns/1ns

module tb_tn1;
    logic [7:0] A, B;
    logic Sel;
    logic [7:0] S;
    logic Co, Ov;

    tn1 dut (.A(A),.B(B),.Sel(Sel),.S(S),.Co(Co),.Ov(Ov));

    initial begin
        $dumpfile("tb_tn1.vcd");
        $dumpvars(0, tb_tn1);

    Sel = 0; A = 8'h00; B = 8'h00; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);
        Sel = 0; A = 8'h28; B = 8'h57; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);
        Sel = 0; A = 8'hFE; B = 8'h07; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);
        Sel = 0; A = 8'h8C; B = 8'hAB; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);
        Sel = 0; A = 8'h01; B = 8'h7F; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);
        Sel = 1; A = 8'h00; B = 8'h00; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);
        Sel = 1; A = 8'h43; B = 8'h25; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);
        Sel = 1; A = 8'h34; B = 8'h8C; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);
        Sel = 1; A = 8'h80; B = 8'h01; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);
        Sel = 1; A = 8'h7F; B = 8'hFF; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);
    Sel = 1; A = 8'h05; B = 8'hFF; #10;
        $display("Time=%0t | Sel=%b A=%h B=%h | S=%h Co=%b Ov=%b", $time, Sel, A, B, S, Co, Ov);

        $finish;
    end
endmodule
```

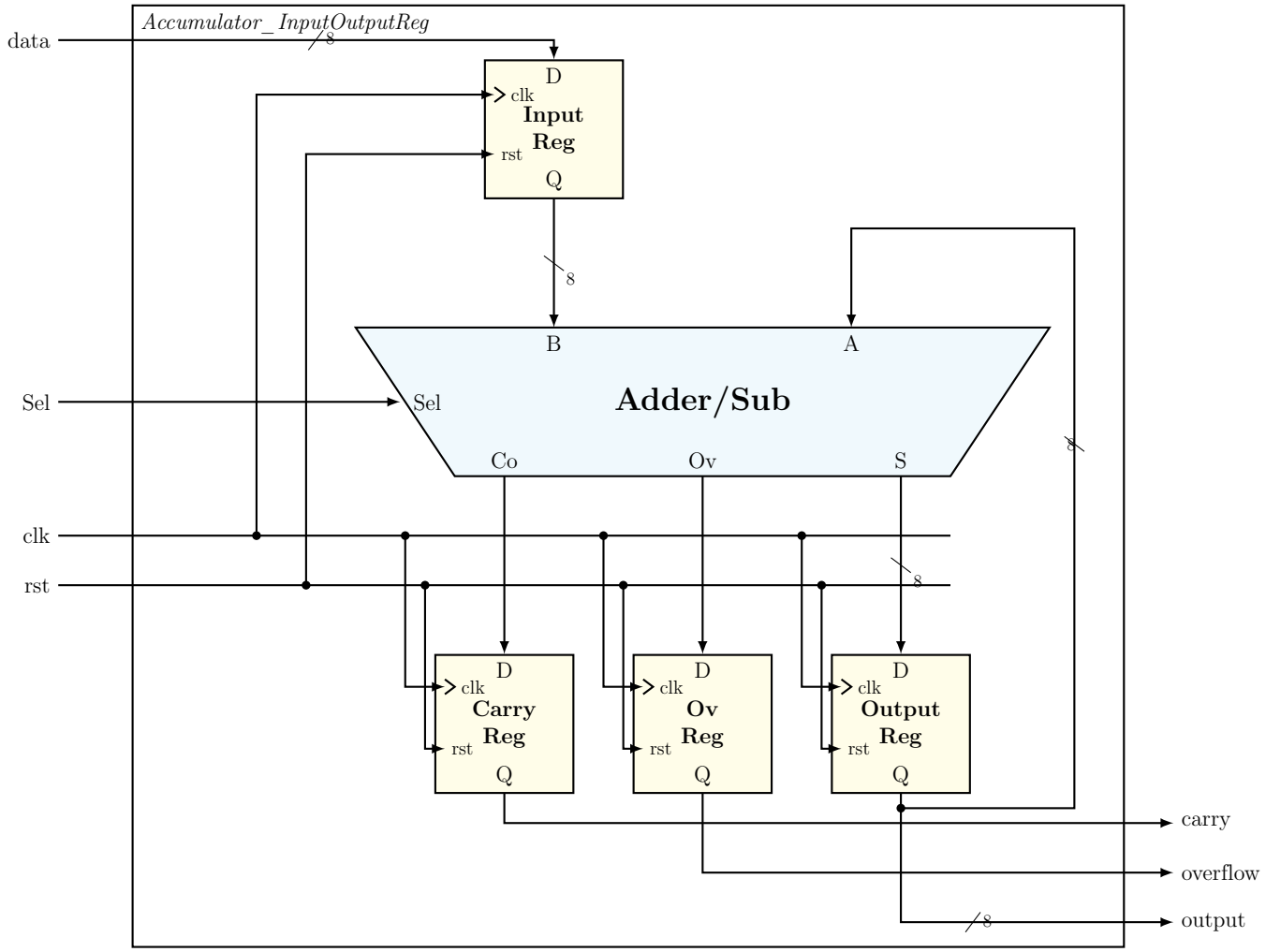
2. Experiment 2: Sequential Circuits - Accumulator

2.1 Sơ đồ thiết kế của mạch.

Sơ đồ bộ tích lũy bao gồm tín hiệu bất đồng bộ reset, ký hiệu rst:



Hình 3: *Sơ đồ thiết kế mạch Accumulator với thanh ghi ở ngõ ra S (Figure 2a).*



Hình 4: *Sơ đồ thiết kế mạch Accumulator với thanh ghi ở ngõ ra S và ngõ vào A (Figure 2b).*

Cách hoạt động của mạch:

- Mạch Accumulator hoạt động như một bộ tích lũy, liên tục thực hiện phép cộng (hoặc trừ) giá trị ngõ vào data với chính giá trị hiện tại của hệ thống (ngõ ra out). Mạch tái sử dụng bộ Adder/Subtractor từ Experiment 1 để thực hiện tính toán.

- Dựa vào sơ đồ thiết kế, ngõ vào data được nối với chân A của bộ cộng, trong khi giá trị phản hồi từ ngõ ra *out* được nối ngược về chân B. Do đó, phép toán thực tế diễn ra bên trong mạch là **B +/- A** chứ không phải A +/- B. Điều này tương đương với biểu thức tích lũy chuẩn của mạch là **out = out +/- data**.

- Kết quả từ bộ cộng *S, Co, Ov* được đưa vào các thanh ghi D-FF. Các thanh ghi này cập nhật trạng thái đồng bộ theo cạnh lên của xung *clk*. Mạch sử dụng tín hiệu reset bất đồng bộ *rst*. Khi *rst = 1*, toàn bộ thanh ghi ngay lập tức bị xóa về 0 mà không cần chờ xung *clk*.

- So sánh sự khác biệt giữa 2 thiết kế: Ở thiết kế trong *Hình 3*, mạch chỉ có thanh ghi ở ngõ ra, còn ở thiết kế trong *Hình 4*, mạch có thêm một thanh ghi *Input Reg* ở ngõ vào *data*. Thanh ghi này giúp đồng bộ hoá dữ liệu đầu vào với xung clock trước khi được đưa vào bộ Adder/Sub, giúp tăng tính ổn định và tránh rủi ro thời gian.

2.2 Mô tả mạch thiết kế bằng ngôn ngữ System Verilog HDL.

Listing 4: `00_src/tn2a.sv`

```
`timescale 1ns/1ns

module tn2a (
    input  logic [7:0] data,
    input  logic       clk,
    input  logic       rst,
    input  logic       Sel,
    output logic [7:0] out,
    output logic       carry,
    output logic       overflow
);
    logic [7:0] next_S;
    logic       next_Co, next_Ov;

    tn1 adder_sub (.A(out),.B(data),.Sel(Sel),.S(next_S),.Co(next_Co),.Ov(next_Ov));
    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            out      <= 8'h00;
            carry    <= 1'b0;
            overflow <= 1'b0;
        end else begin
            out      <= next_S;
            carry    <= next_Co;
            overflow <= next_Ov;
        end
    end
endmodule
```

Listing 5: `00_src/tn2b.sv`

```
`timescale 1ns/1ns

module tn2b (
    input  logic [7:0] data,
    input  logic       clk,
    input  logic       rst,
    input  logic       Sel,
    output logic [7:0] out,
    output logic       carry,
    output logic       overflow
);
    logic [7:0] reg_data;
    logic [7:0] next_S;
    logic       next_Co, next_Ov;

    tn1 adder_sub (
        .A(out),
        .B(reg_data),
        .Sel(Sel),
        .S(next_S),
        .Co(next_Co),
        .Ov(next_Ov)
    );

    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            reg_data <= 8'h00;
            out      <= 8'h00;
            carry    <= 1'b0;
            overflow <= 1'b0;
        end else begin
            reg_data <= data;
            out      <= next_S;
            carry    <= next_Co;
            overflow <= next_Ov;
        end
    end
endmodule
```

2.3 Hoàn thành giản đồ xung dựa theo thiết kế trên.

Hình 5: *Giản đồ thời gian mô phỏng mạch Accumulator theo thiết kế 2a.*

Hình 6: *Giản đồ thời gian mô phỏng mạch Accumulator theo thiết kế 2b.*

Giải thích từng trường hợp:

Trong đoạn mã testbench kiểm chứng, tín hiệu Sel = 0 được giữ cố định, do đó hệ thống hoạt động hoàn toàn như một bộ cộng dồn. Áp dụng logic B + A đã phân tích, phép toán tại mỗi chu kỳ là **out = out + data**

- Quá trình khởi tạo: Khi rst = 1, toàn bộ ngõ ra output = 0x00, carry = 0, overflow = 0. Khi rst = 0, mạch bắt đầu tích lũy.
- Ở chu kì 1, khi data=0x70: **out = 0x00 + 0x70 = 0x70**. Kết quả nằm trong miền biểu diễn hợp lệ → carry = 0, overflow = 0.
- Ở chu kì 2, có data=0x25: Giá trị hiện tại 0x70 (Dương 112) + 0x25 (Dương 37) = 0x95. Trong hệ biểu diễn bù 2 (8-bit), 0x95 tương đương với giá trị Âm 107. Cộng hai số dương sinh ra số âm, vi phạm logic dấu → Cờ tràn overflow = 1. Do không vượt quá 255 nên carry = 0.
- Chu kỳ 3, data = 0x5A: kết quả trước có 0x95 (Âm) + 0x5A (Dương) = 0xEF (Âm). Phép cộng hai số trái dấu luôn an toàn → Cờ overflow = 0, carry = 0.
- Chu kỳ 4, data = 0x30: có kết quả trước 0xEF + 0x30 = 0x11F. Ngõ ra S bị cắt ngọn, chỉ lưu giữ 8-bit cuối là 0x1F. Phép toán vượt quá ngưỡng tối đa của số không dấu (255) → Sinh cờ nhớ carry = 1. Về mặt có dấu: Âm (-17) + Dương (48) = Dương (31), hợp logic → overflow = 0.
- Chu kỳ 5, data = 0x7F: 0x1F (Dương 31) + 0x7F (Dương 127) = 0x9E (Âm 98). Cộng hai số dương sinh ra số âm → Bật cờ overflow = 1, carry = 0.
- Chu kỳ 6, data = 0x90: 0x9E (Âm) + 0x90 (Âm) = 0x12E. Mạch giữ lại 0x2E (số Dương). Phép toán đồng thời vượt ngưỡng biểu diễn 8-bit → carry = 1, và vi phạm quy tắc cộng hai số âm sinh ra số dương → overflow = 1.

2.4 Viết Testbench kiểm chứng thiết kế dựa trên giản đồ xung.

Listing 6: `01_tb/tb_tn2a.sv`

```
`timescale 1ns/1ns

module tb_tn2a;
    logic [7:0] data;
    logic       clk;
    logic       rst;
    logic       Sel;
    logic [7:0] out;
    logic       carry;
    logic       overflow;

    tn2a dut (.data(data),.clk(clk),.rst(rst),.Sel(Sel),.out(out),.carry(carry),.overflow(overflow));
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

   initial begin
        $dumpfile("tb_tn2a.vcd");
        $dumpvars(0, tb_tn2a);

        rst = 0; Sel = 0; data = 8'h00; #5 rst = 1; #5 rst = 0;
        $display("--- Bat dau mo phong Accumulator (Sel=0: Add, Sel=1: Sub) ---");
        data = 8'h70; #10;
        data = 8'h25; #10;
        data = 8'h5A; #10;
        data = 8'h30; #10;
        data = 8'h1F; #10;
        data = 8'h90; #10;
        data = 8'hE8; #10;
        data = 8'h80; #10;
        #15;
```

```
        $display("--- Ket thuc mo phong ---");
        $finish;
    end
    initial begin
        $monitor("Time=%0t | rst=%b | Sel=%b | Data_In=%h | Accumulator_Out=%h | Co=%b | Ov=%b",
                 $time, rst, Sel, data, out, carry, overflow);
    end

endmodule
```

Listing 7: `01_tb/tb_tn2b.sv`

```
`timescale 1ns/1ns
module tb_tn2b;
    logic [7:0] data;
    logic       clk;
    logic       rst;
    logic       Sel;
    logic [7:0] out;
    logic       carry;
    logic       overflow;

    tn2b dut (.data(data),.clk(clk),.rst(rst),.Sel(Sel),.out(out),.carry(carry),.overflow(overflow));
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
    initial begin
        $dumpfile("tb_tn2b.vcd");
        $dumpvars(0, tb_tn2b);

        rst = 0; Sel = 0; data = 8'h00; #5 rst = 1; #5 rst = 0;
        $display("--- Bat dau mo phong Accumulator (Sel=0: Add, Sel=1: Sub) ---");
        data = 8'h70; #10;
        data = 8'h25; #10;
        data = 8'h5A; #10;
        data = 8'h30; #10;
        data = 8'h1F; #10;
        data = 8'h90; #10;
        data = 8'hE8; #10;
        data = 8'h80; #10;
        #15;
        $display("--- Ket thuc mo phong ---");
        $finish;
    end

   initial begin
        $monitor("Time=%0t | rst=%b | Sel=%b | Data_In=%h | Accumulator_Out=%h | Co=%b | Ov=%b",
                 $time, rst, Sel, data, out, carry, overflow);
    end

endmodule
```

2.5 So sánh giữa hai thiết kế mạch

Mặc dù cả hai mạch đều thực hiện cùng một chức năng tích lũy, việc chèn thêm thanh ghi ngõ vào ở thiết kế 2(b) tạo ra sự đánh đổi rõ rệt về hiệu suất:

- Về cấu trúc và diện tích: Thiết kế 2(a) tiết kiệm phần cứng hơn vì chỉ sử dụng các thanh ghi ở ngõ ra. Trong khi đó, thiết kế 2(b) tốn kém diện tích hơn do phải tốn thêm 8 D-FF để xây dựng thanh ghi 8-bit cho ngõ vào A.

- Về độ trễ: Thiết kế 2(a) phản hồi nhanh hơn. Dữ liệu ngõ vào đi trực tiếp qua bộ cộng tổ hợp và cập nhật ra kết quả ngay tại sườn clock tiếp theo. Ngược lại, thiết kế 2(b) có độ trễ lớn hơn 1 chu kỳ clock, vì dữ liệu phải tốn một nhịp nạp vào Input Reg rồi mới được đưa vào tính toán.

- Về tần số hoạt động và Timing: Đây là ưu điểm vượt trội của thiết kế 2(b). Ở mạch 2(a), đường găng bị kéo dài từ nguồn cấp dữ liệu bên ngoài → đi xuyên qua bộ cộng → tới thanh ghi ngõ ra. Nếu thiết bị ngoài cấp dữ liệu trễ, mạch 2(a) rất dễ bị vi phạm thời gian thiết lập. Ngược lại, thiết kế 2(b) áp dụng nguyên lý đường ống cơ bản: cô lập khối logic tổ hợp vào giữa hai ranh giới thanh ghi. Điều này giúp cắt ngắn đường găng, từ đó hệ thống có thể hoạt động ổn định ở tần số xung clock cao hơn rất nhiều.

3. Experiment 3: FSM Implementation, Logic-Expression Style

3.1 Phương trình trạng thái và ngõ ra

Dựa vào lưu đồ chuyển trạng thái, ta thiết lập được các phương trình Boolean cho logic Next-state và Output.

Đối với mã One-hot (Sử dụng 9 biến trạng thái từ Q_A đến Q_I):

- **Phương trình trạng thái tiếp theo:**

$$
\begin{aligned}
D_A &= 0 \quad \text{(Chỉ đạt được thông qua tín hiệu Reset)} \\
D_B &= \overline{w} \cdot (Q_A + Q_F + Q_G + Q_H + Q_I) \\
D_C &= \overline{w} \cdot Q_B \\
D_D &= \overline{w} \cdot Q_C \\
D_E &= \overline{w} \cdot (Q_D + Q_E) \quad \text{(Self-loop khi } w = 0\text{)} \\
D_F &= w \cdot (Q_A + Q_B + Q_C + Q_D + Q_E) \\
D_G &= w \cdot Q_F \\
D_H &= w \cdot Q_G \\
D_I &= w \cdot (Q_H + Q_I) \quad \text{(Self-loop khi } w = 1\text{)}
\end{aligned}
$$

- **Phương trình ngõ ra:** $z = Q_E + Q_I$

Đối với mã Binary: Do sử dụng 4 biến trạng thái ($Y_3Y_2Y_1Y_0$), việc rút gọn bằng bìa Karnaugh sẽ tạo ra các phương trình Boolean rất phức tạp. Do đó, phương trình next-state cho mã Binary sẽ được biểu diễn trực tiếp dưới dạng biểu thức điều kiện (Logic-expression/MUX logic) trong phần mã nguồn SystemVerilog. Phương trình ngõ ra: $z = \overline{Y_3}Y_2\overline{Y_1Y_0} + Y_3\overline{Y_2Y_1Y_0}$.

3.2 Mô tả mạch thiết kế bằng ngôn ngữ System Verilog HDL.

Listing 8: `00_src/fsm_binary.sv`

```
`timescale 1ns/1ns
module fsm_binary (
    input  logic clk,
    input  logic rst,
    input  logic w,
    output logic z
);

    logic [3:0] state, next_state;

    localparam logic [3:0] A = 4'b0000, B = 4'b0001,C = 4'b0010, D = 4'b0011, E = 4'b0100,
    F = 4'b0101, G = 4'b0110, H = 4'b0111, I = 4'b1000;

    always_ff @(posedge clk) begin
        if (rst)
            state <= A;
        else
            state <= next_state;
    end

    assign next_state =
        (state == A) ? (w ? F : B) :
        (state == B) ? (w ? F : C) :
        (state == C) ? (w ? F : D) :
        (state == D) ? (w ? F : E) :
        (state == E) ? (w ? F : E) :
```

```
        (state == F) ? (w ? G : B) :
        (state == G) ? (w ? H : B) :
        (state == H) ? (w ? I : B) :
        (state == I) ? (w ? I : B) :
        A;

    assign z = (state == E) || (state == I);

endmodule
```

Cách hoạt động: Trong thiết kế này, hệ thống sử dụng thanh ghi 4-bit để lưu trữ 9 trạng thái (từ A đến I). Mạch được chia làm 3 khối logic rõ ràng:

- Khối lưu trữ trạng thái: Sử dụng khối `always_ff` để cập nhật trạng thái hiện tại (`state`) bằng trạng thái tiếp theo (`next_state`) tại mỗi sườn lên của xung `clk` . Khi tín hiệu `rst` được kích hoạt, hệ thống lập tức trở về trạng thái khởi tạo A .
- Khối logic chuyển trạng thái: Do yêu cầu sử dụng phong cách `logic-expression`, mạch không sử dụng khối `always_comb` hay lệnh `if/case`. Thay vào đó, toàn bộ logic chuyển trạng thái được mô tả bằng một chuỗi toán tử điều kiện 3 ngôi (`?:`) nối tiếp nhau trong một lệnh `assign` duy nhất. Cấu trúc này khi tổng hợp (synthesis) sẽ tạo ra một mạng lưới các bộ dồn kênh (Multiplexer) xếp tầng khổng lồ ở mức cổng logic.
- Khối logic ngõ ra: Ngõ ra z là một mạch tổ hợp đơn giản, kiểm tra xem `state` hiện tại có đang bằng `E` (phát hiện 4 bit 0) hoặc bằng `I` (phát hiện 4 bit 1) hay không để xuất mức logic 1 .

Listing 9: `00_src/fsm_onehot.sv`

```
`timescale 1ns/1ns
module fsm_onehot (
    input  logic clk,
    input  logic rst,
    input  logic w,
    output logic z
);
    // state[0]=A, [1]=B, [2]=C, [3]=D, [4]=E
    // state[5]=F, [6]=G, [7]=H, [8]=I
    logic [8:0] state, next_state;
    always_ff @(posedge clk) begin
        if (rst)
            state <= 9'b000000001;
        else
            state <= next_state;
    end
    // Next-state logic (direct Boolean equations from state diagram)
    assign next_state[0] = 1'b0;
    assign next_state[1] = ~w & (state[0] | state[5] | state[6] | state[7] | state[8]);
    assign next_state[2] = ~w & state[1];
    assign next_state[3] = ~w & state[2];
    assign next_state[4] = ~w & (state[3] | state[4]);   // overlapping at E
    assign next_state[5] =  w & (state[0] | state[1] | state[2] | state[3] | state[4]);
    assign next_state[6] =  w & state[5];
    assign next_state[7] =  w & state[6];
    assign next_state[8] =  w & (state[7] | state[8]);   // overlapping at I

    // Output: z=1 in state E (bit 4) or state I (bit 8)
    assign z = state[4] | state[8];

endmodule
```

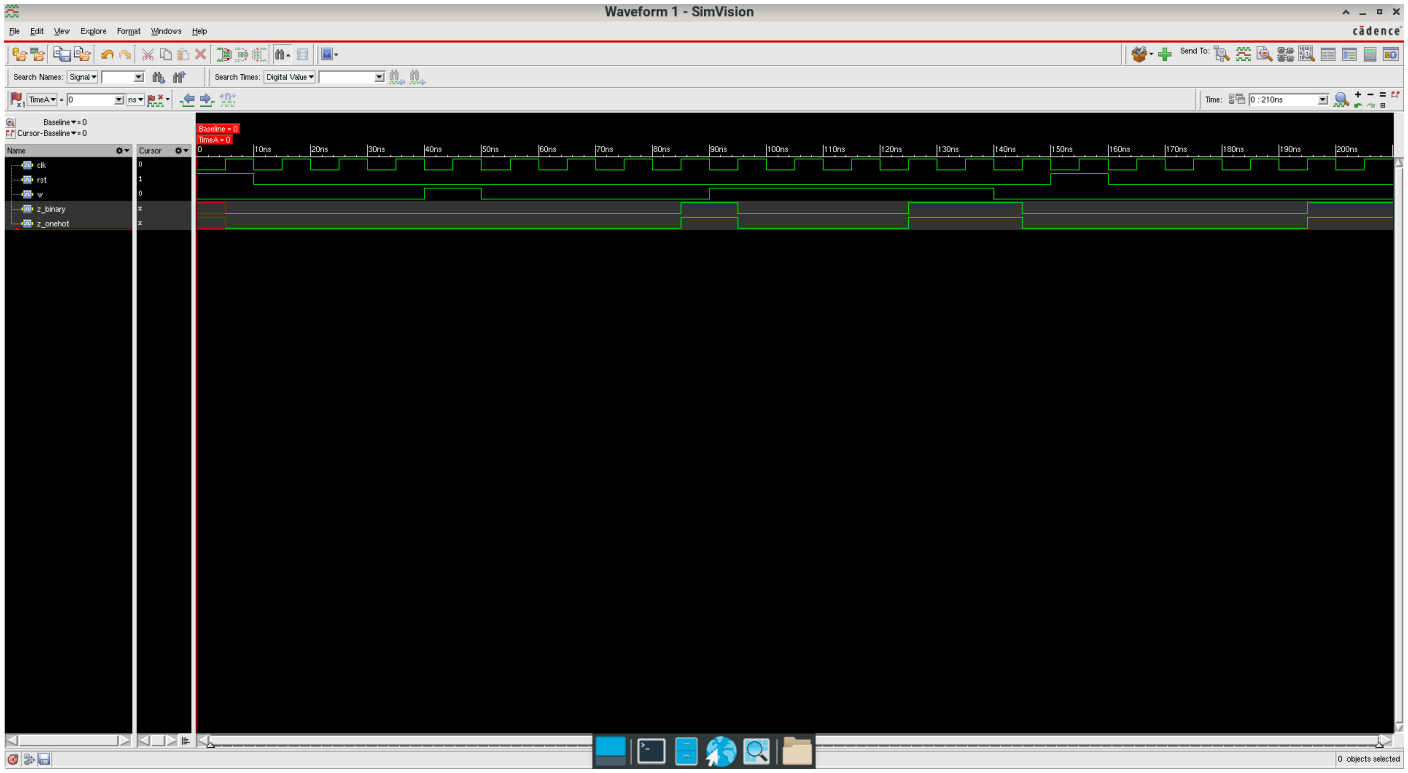
Cách hoạt động: Khác với Binary, mã hóa One-hot sử dụng đến 9 bit (tương ứng với 9 Flip-flop) để biểu diễn 9 trạng thái . Tại bất kỳ thời điểm nào, chỉ có duy nhất 1 bit mang giá trị 1.

- Khối lưu trữ trạng thái: Tương tự như Binary, khối `always_ff` làm nhiệm vụ chốt trạng thái theo xung `clk` . Tuy nhiên, khi có tín hiệu `rst`, trạng thái A được khởi tạo bằng chuỗi bit `9'b000000001` (chỉ bit số 0 bật lên 1) .

- Khối logic chuyển trạng thái: Đây là điểm sáng của thiết kế One-hot. Mỗi bit của `next_state` (từ bit 0 đến bit 8) được tính toán riêng biệt thông qua một lệnh `assign` độc lập . Các lệnh này tương ứng trực tiếp với các phương trình Boolean đã được rút gọn (Ví dụ: bit `next_state[2]` chỉ bằng 1 khi ngõ vào $w = 0$ và hệ thống đang ở `state[1]`) . Cấu trúc này giúp mạch tổng hợp ra các cổng AND/OR rất nhỏ gọn, chạy ở tần số rất cao vì không bị vướng bộ MUX phức tạp như mã Binary.

- Khối logic ngõ ra: Rất tối ưu, hệ thống không cần dùng đến các bộ so sánh. Mạch chỉ cần dùng một cổng OR duy nhất để lấy tổng của bit số 4 (`state[4]` tương ứng với E) và bit số 8 (`state[8]` tương ứng với I).

3.3 Hoàn thành giản đồ xung dựa theo thiết kế trên.



Hình 7: *Kết quả giản đồ xung của 2 loại mã binary coding và one-hot codes*

Giải thích:

Dựa vào giản đồ thời gian mô phỏng, ta có thể thấy cả hai bộ mã hóa Binary và One-hot đều hoạt động song song và cho ra đáp ứng ngõ ra z hoàn toàn trùng khớp nhau dưới cùng một chuỗi tín hiệu kích thích w. Cụ thể qua các giai đoạn:

- **Khởi tạo:** Khi tín hiệu $rst = 1$, FSM bị cưỡng bức về trạng thái ban đầu A (0000 đối với Binary và 000000001 đối với One-hot). Ngõ ra $z = 0$.

- **Phát hiện chuỗi 4 bit 0:** Khi $w = 0$ liên tục trong 4 chu kỳ xung nhịp, FSM lần lượt dịch chuyển qua các trạng thái $A \to B \to C \to D \to E$. Ngay tại trạng thái E, ngõ ra z bật lên 1, báo hiệu đã nhận diện thành công chuỗi 4 bit 0.

- **Tính năng chồng chéo:** Ở chu kỳ tiếp theo, nếu w vẫn tiếp tục bằng 0, FSM tự lặp lại tại trạng thái E ($E \to E$) và ngõ ra z vẫn duy trì mức 1.

- **Phát hiện chuỗi 4 bit 1:** Khi w chuyển sang 1, chuỗi đếm 0 bị ngắt, mạch nhảy sang trạng thái F. Tiếp tục giữ $w = 1$ trong 4 chu kỳ, mạch đi theo nhánh $F \to G \to H \to I$. Tại trạng thái I, ngõ ra $z = 1$. Tính năng chồng chéo tiếp tục hoạt động đúng nếu w giữ mức 1 ở các nhịp sau đó.
- **Chuỗi bị ngắt quãng:** Khi đang đếm dở dang (ví dụ đang ở C, tức đếm được hai bit 0), nếu tín hiệu w đột ngột đảo mức thành 1, hệ thống lập tức hủy chuỗi đang đếm và chuyển hướng sang nhánh đếm mới (nhảy sang F). Ngõ ra z giữ nguyên mức 0, chứng minh mạch không bị nhận diện sai.

Kết luận: Giản đồ xung chứng minh mạch hoạt động chính xác theo đặc tả thiết kế.

3.4 Viết Testbench kiểm chứng thiết kế dựa trên giản đồ xung.

Listing 10: `01_tb/tb_fsm.sv`

```
`timescale 1ns/1ns

module tb_fsm();
    logic clk;
    logic rst;
    logic w;
    logic z_onehot;
    logic z_binary;
    fsm_onehot u1 (.clk(clk), .rst(rst), .w(w), .z(z_onehot));
    fsm_binary u2 (.clk(clk), .rst(rst), .w(w), .z(z_binary));


    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        $dumpfile("tb_fsm.vcd");
        $dumpvars(0,tb_fsm);
        w = 0; rst = 1; #10;
        rst = 0; #30;
        w = 1; #10;
        w = 0; #40;
        w = 1; #50;
        w = 0; #10;
        rst = 1; #10;
        rst = 0; #50;
        $display("Simulation Finished!");
        $finish;
    end

    initial begin
        $monitor("Time=%0t | rst=%b | w=%b | z_onehot=%b | z_binary=%b", $time, rst, w, z_onehot,
        z_binary);
    end
endmodule
```

4. Experiment 4: FSM Implementation, Behavioral Style

Ở phần này, FSM nhận diện chuỗi được thiết kế lại bằng phong cách mô tả hành vi (Behavioral Style) của SystemVerilog. Thay vì tự tính toán các phương trình logic, ta sử dụng khối `always_comb` kết hợp với lệnh `case` để mô tả trực tiếp luồng chuyển trạng thái.

4.1 Mô tả mạch thiết kế bằng ngôn ngữ System Verilog HDL.

Cả hai phương pháp mã hóa One-hot và Binary đều cho cùng một hành vi đáp ứng ngõ ra z theo ngõ vào w.

1. Mã hóa Binary (Behavioral Style):

Listing 11: 00_src/fsm_binary_behav.sv

```
`timescale 1ns/1ns
module fsm_binary_behav (
    input logic clk,
    input logic rst,
    input logic w,
    output logic z
);
    typedef enum logic [3:0] {
        A = 4'd0,
        B = 4'd1,
        C = 4'd2,
        D = 4'd3,
        E = 4'd4,
        F = 4'd5,
        G = 4'd6,
        H = 4'd7,
        I = 4'd8
    } state_t;

    state_t present_state, next_state;
    always_ff @(posedge clk, posedge rst) begin
        if (rst)
            present_state <= A;
        else
            present_state <= next_state;
    end

    always_comb begin
        next_state = present_state;
        case (present_state)
            A: if (w) next_state = F; else next_state = B;
            B: if (w) next_state = F; else next_state = C;
            C: if (w) next_state = F; else next_state = D;
            D: if (w) next_state = F; else next_state = E;
            E: if (w) next_state = F; else next_state = E;
            F: if (w) next_state = G; else next_state = B;
            G: if (w) next_state = H; else next_state = B;
            H: if (w) next_state = I; else next_state = B;
            I: if (w) next_state = I; else next_state = B;
            default: next_state = A;
        endcase
    end

    assign z = (present_state == E) || (present_state == I);
endmodule
```

2. Mã hóa One-hot (Behavioral Style):

Listing 12: 00_src/fsm_onehot_behav.sv

```
`timescale 1ns/1ns
module fsm_onehot_behav (
    input logic clk,
    input logic rst,
    input logic w,
    output logic z
);
    typedef enum logic [8:0] {
        A = 9'b000000001,
        B = 9'b000000010,
        C = 9'b000000100,
        D = 9'b000001000,
        E = 9'b000010000,
        F = 9'b000100000,
        G = 9'b001000000,
        H = 9'b010000000,
        I = 9'b100000000
    } state_t;

    state_t present_state, next_state;
    always_ff @(posedge clk, posedge rst) begin
        if (rst)
            present_state <= A;
        else
            present_state <= next_state;
    end

    always_comb begin
        next_state = present_state;
        case (present_state)
            A: if (w) next_state = F; else next_state = B;
            B: if (w) next_state = F; else next_state = C;
            C: if (w) next_state = F; else next_state = D;
            D: if (w) next_state = F; else next_state = E;
            E: if (w) next_state = F; else next_state = E;
            F: if (w) next_state = G; else next_state = B;
            G: if (w) next_state = H; else next_state = B;
            H: if (w) next_state = I; else next_state = B;
            I: if (w) next_state = I; else next_state = B;
            default: next_state = A;
        endcase
    end

    assign z = (present_state == E) || (present_state == I);
endmodule
```

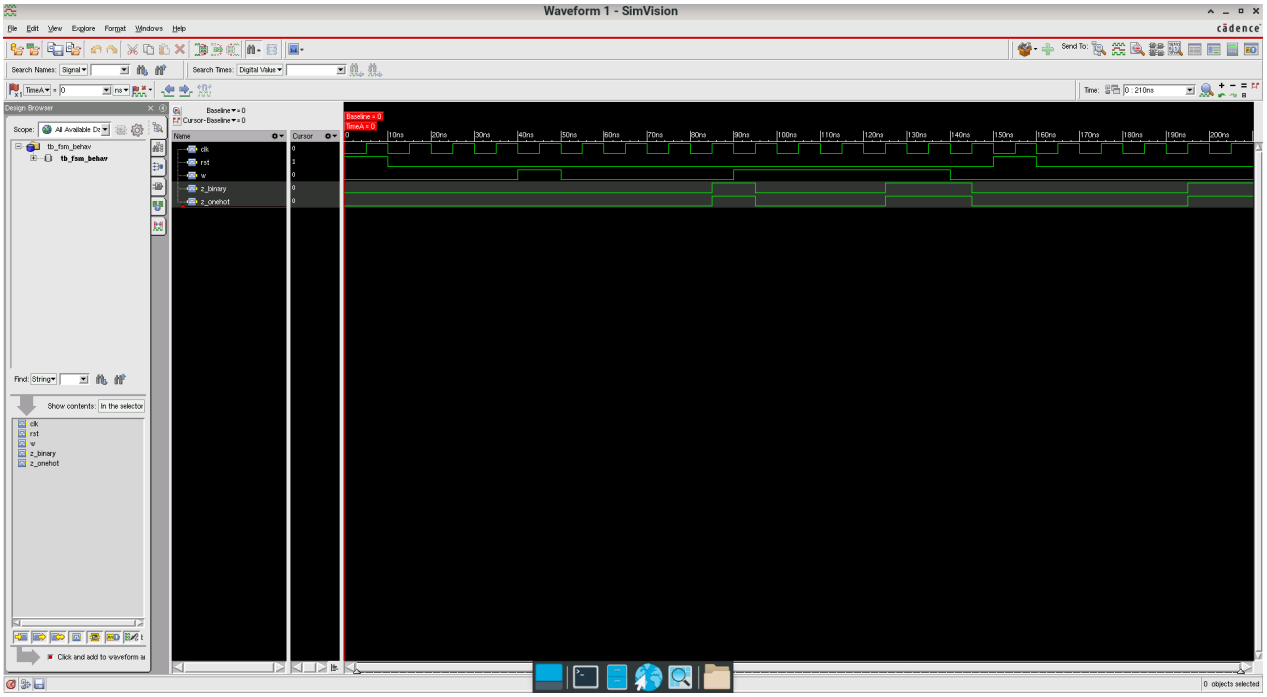
4.2 Hoàn thành giản đồ xung dựa theo thiết kế trên.

Về mặt nhận diện chuỗi, mạch ở Experiment 4 thực hiện chức năng hoàn toàn giống với Experiment 3, do đó đáp ứng của ngõ ra z đối với dữ liệu w là trùng khớp. Tuy nhiên, có một sự khác biệt về bản chất phần cứng được thể hiện trên giản đồ thời gian: **Hành vi của tín hiệu Reset**.

- Ở **Experiment 3**, hệ thống sử dụng **Reset Đồng Bộ** (`always_ff @(posedge clk)`). Nếu tín hiệu rst được kích hoạt giữa chu kỳ, hệ thống bắt buộc phải chờ đến sườn lên của xung clk tiếp theo mới đưa trạng thái về A.

- Ở **Experiment 4**, thiết kế sử dụng **Reset Bất Đồng Bộ** (`always_ff @(posedge clk, posedge rst)`). Khi quan sát kỹ giản đồ xung, ta sẽ thấy ngay khi tín hiệu rst chuyển lên mức 1, toàn bộ trạng thái (`present_state`) lập tức bị ép về trạng thái khởi tạo A mà không cần quan tâm đến sườn xung clk.

Tính năng này giúp hệ thống ở Experiment 4 phản ứng an toàn và tức thời hơn trước các sự cố lỗi toàn cục cần khôi phục (hardware panic reset).



Hình 8: *Giản đồ thời gian FSM (Behavioral) - Mã Binary.*

4.3 Viết Testbench kiểm chứng thiết kế dựa trên giản đồ xung.

Vì FSM ở phần này có cùng chức năng, số lượng trạng thái và các cổng giao tiếp (w, clk, rst, z) với Experiment 3, ta hoàn toàn có thể tái sử dụng lại các file Testbench đã viết ở phần trước.

Để tiến hành mô phỏng, ta chỉ cần thay đổi tên module được khởi tạo (instantiation) trong file Testbench từ `fsm_binary` thành `fsm_behav_binary` (và tương tự cho module One-hot).

Listing 13: 00_src/tb_fsm_behav.sv

```
`timescale 1ns/1ns

module tb_fsm_behav();
    logic clk;
    logic rst;
    logic w;
    logic z_onehot;
    logic z_binary;
    fsm_onehot_behav u1 (.clk(clk), .rst(rst), .w(w), .z(z_onehot));
    fsm_binary_behav u2 (.clk(clk), .rst(rst), .w(w), .z(z_binary));


    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        $dumpfile("tb_fsm_behav.vcd");
        $dumpvars(0,tb_fsm_behav);
        w = 0; rst = 1; #10;
        rst = 0; #30;
        w = 1; #10;
        w = 0; #40;
        w = 1; #50;
```

```
        w = 0; #10;
        rst = 1; #10;
        rst = 0; #50;
        $display("Simulation Finished!");
        $finish;
    end

    initial begin
        $monitor("Time=%0t | rst=%b | w=%b | z_onehot=%b | z_binary=%b", $time, rst, w, z_onehot,
        z_binary);
    end
endmodule
```